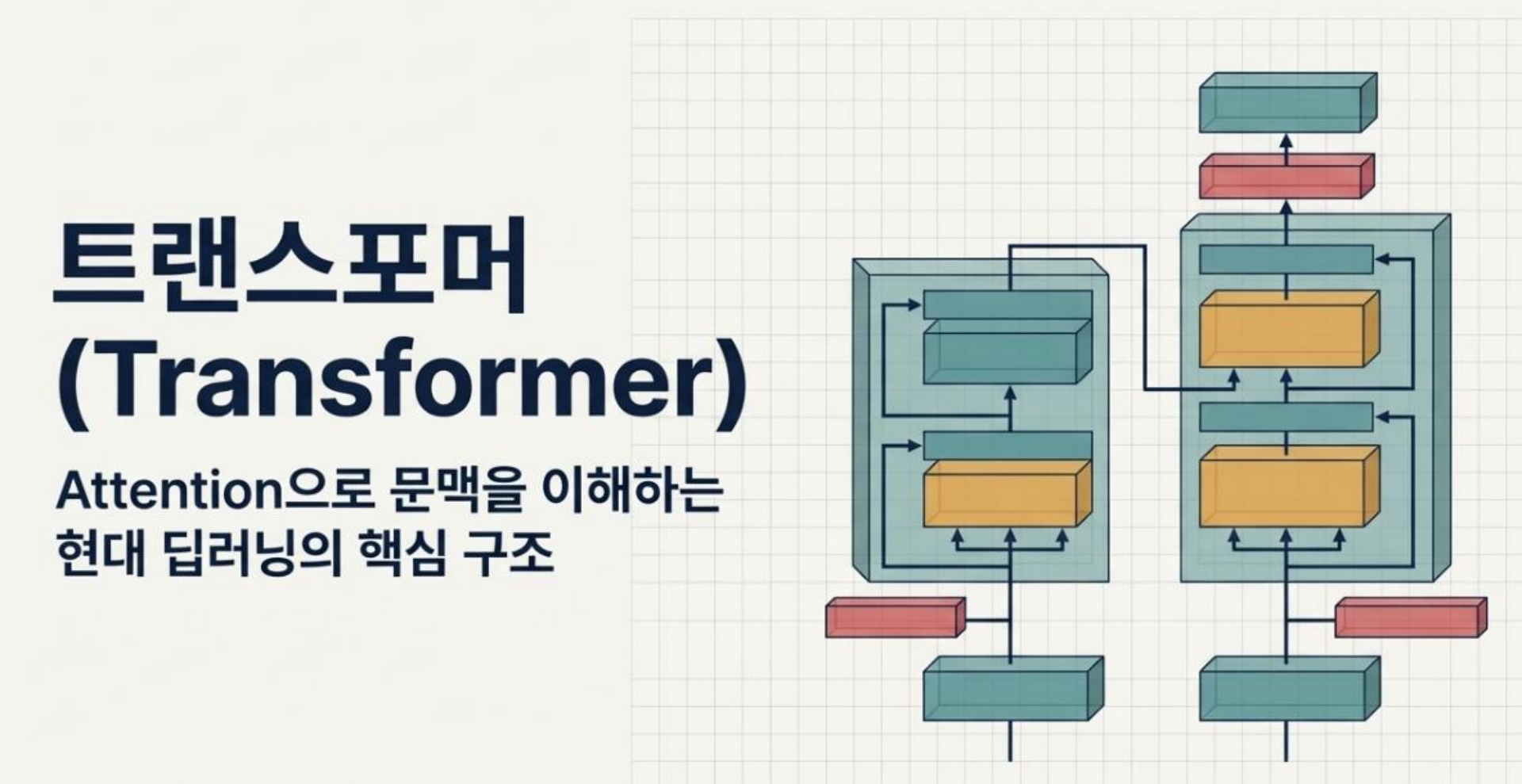
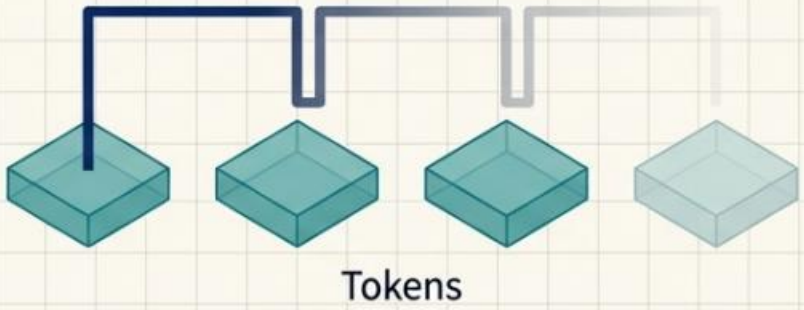
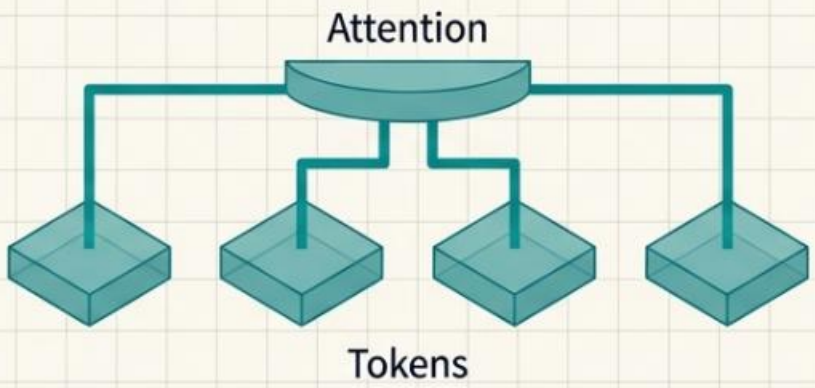



**트랜스포머(Transformer)는**
**문장 안의 모든 단어 관계를 동시에 계산하여 의미를 이해하는 딥러닝 모델 구조**

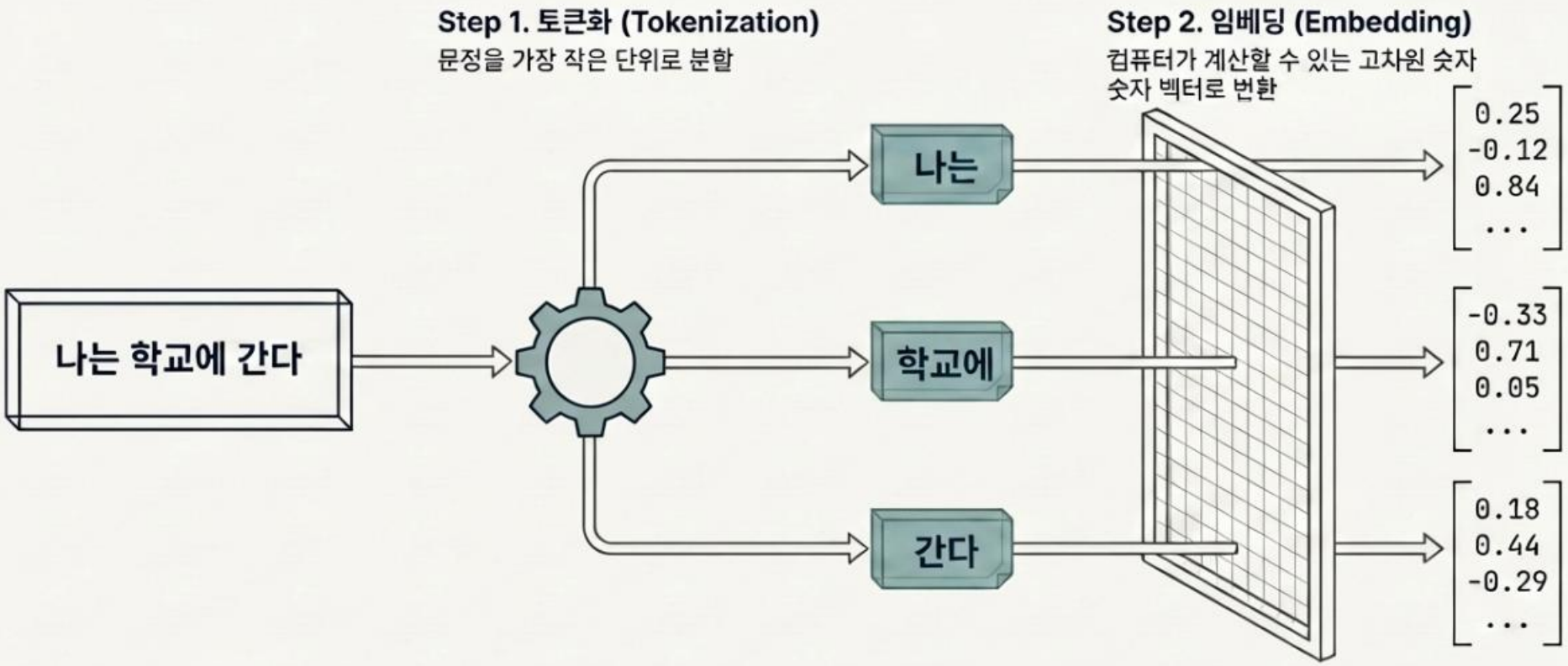
# RNN의 한계와 트랜스포머의 등장

- Attention: 문장 내 각 단어가 다른 모든 단어와 어떤 관련성이 있는지 계산하여 중요한 정보에 가중치를 두는 메커니즘

| 과거: 순차 처리 방식 (RNN) | 혁신: 병렬 처리 방식 (Transformer) |
| --- | --- |
|   |   |
| • **순차 처리:** 단어를 한 번에 하나씩<br>• **장기 의존성 문제:** 정보가 뒤로 갈수록 약해짐<br>• **학습 속도 저하:** 병렬 계산 불가 | • **전체 문장 병렬 처리:** 문장 전체를 한꺼번에 계산<br>• **직접 연결:** 거리 상관없이 연관성 파악<br>• **압도적 속도 향상:** 동시 처리 가능 |

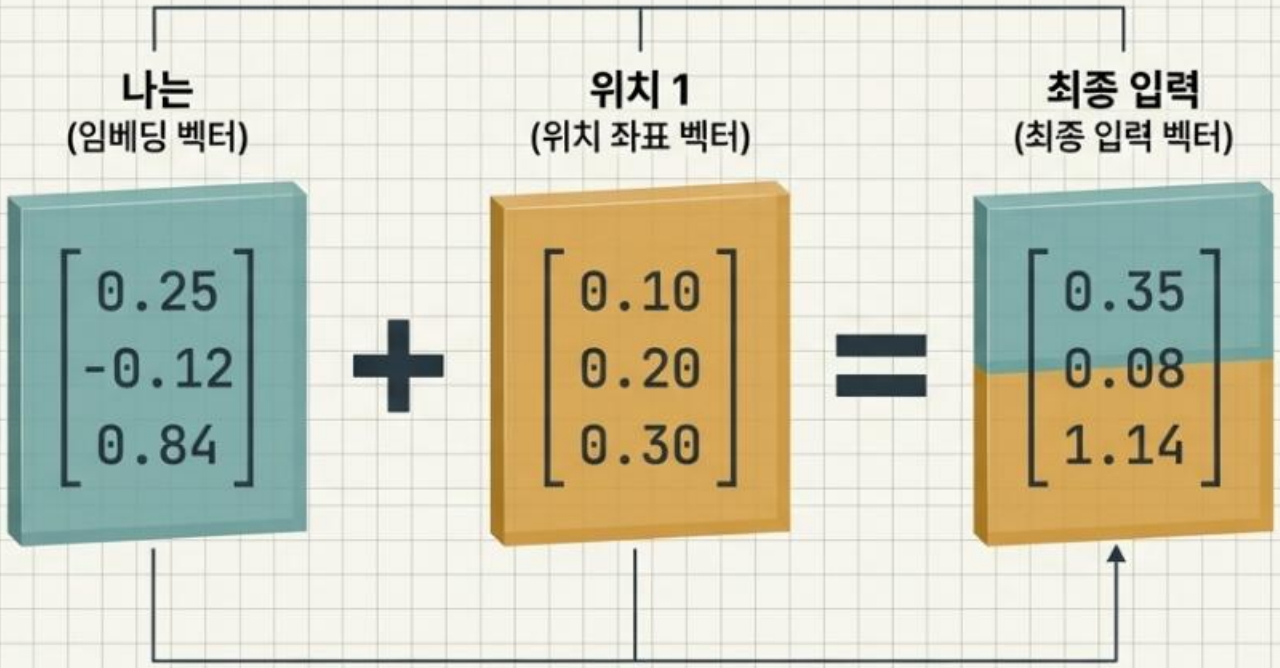
**패러다임의 전환: '기억을 이어가는 방식'에서 '전체를 보며 관련성을 계산하는 방식'으로 발전**

# 데이터 준비: 토큰(Token)과 임베딩(Embedding)



# 위치 인코딩 (Positional Encoding): 단어에 좌표 부여하기

트랜스포머는 모든 단어를 동시에 병렬로 읽기 때문에, 단어의 위치(시간) 정보를 스스로 알 수 없습니다.
이를 해결하기 위해 각 단어의 벡터에 좌표값을 더해줍니다.




**왜 중요할까요?**

"철수가 영희를 좋아한다"
≠
"영희가 철수를 좋아한다"

구성 단어는 같지만 위치 정보가 추가되어 완전히 다른 문장으로 인식됩니다.

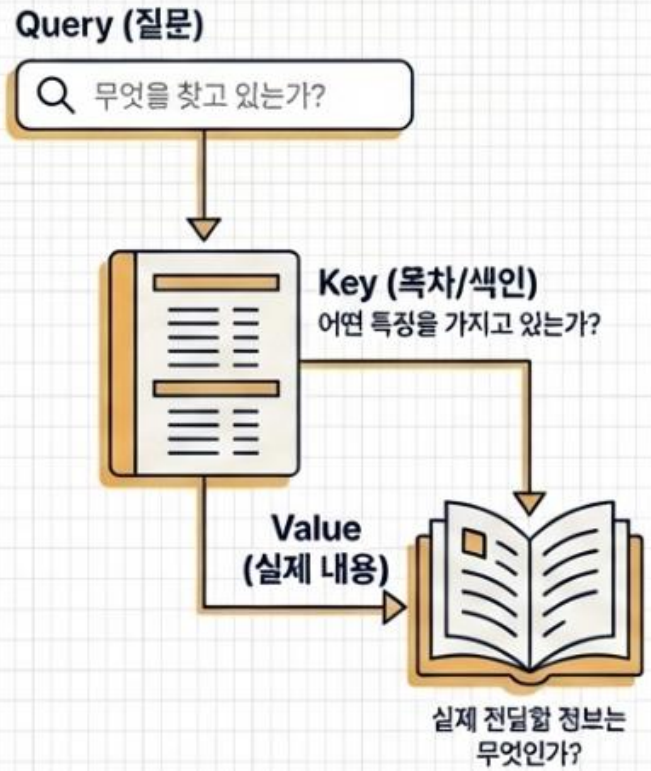
# 혁신의 핵심: Self-Attention (셀프 어텐션)

| 질문: | 메커니즘: |
| --- | --- |
| 현재 단어가 문장 안의 어떤 단어를 더 주의 깊게 볼 것인가? | 단어를 고립된 상태로 보지 않고, 한 문장 내의 모든 단어가 서로를 참조하여 가중치(중요도)를 계산합니다. 이 과정을 통해 단순한 단어 벡터가 '문맥이 반영된 풍부한 벡터'로 진화합니다. |

**Attention은 단어가 다른 단어를 얼마나 참고할지 계산하는 것**입니다.

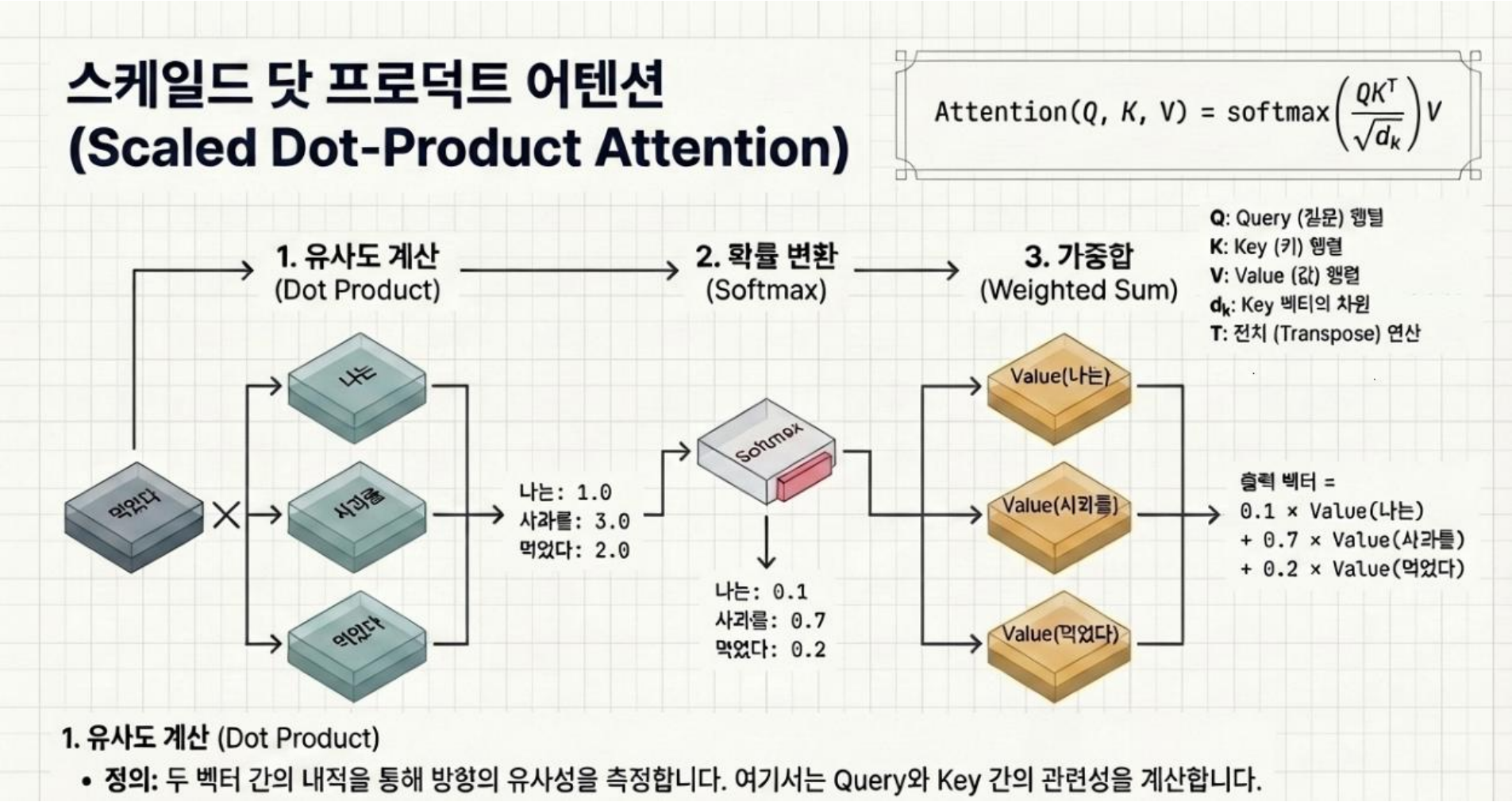
# 엔진의 작동 원리: Query, Key, Value

## 예) 나는 사과를 먹었다




| Name | Role | Technical Definition | Example |
|---|---|---|---|
| Query (Q) | 무엇을 찾고 있는가? | 현재 이해하려는 기준 단어 | 먹었다 |
| Key (K) | 어떤 특징을 가지고 있는가? | Query와 관련성을 비교할 대상 단어들 | 나는, 사과를, 먹었다 |
| Value (V) | 실제 전달할 정보는 무엇인가? | 각 단어가 가지고 있는 실제 정보 벡터 | 나는, 사과를, 먹었다 |

트랜스포머는 각 토큰에서 Q(Query), K(Key), V(Value)를 만듭니다.




- Dot Product(내적)의 값이 클수록 이 정보는 나랑 관련이 깊다라는 뜻입니다.

- Scaled(크기 조정) : 유사도 점수가 커지면 나중에 계산 오류(경사 소실)가 생길 수 있습니다. 그래서 벡터의 크기로 나누어 숫자를 적당히 작게 조절합니다.

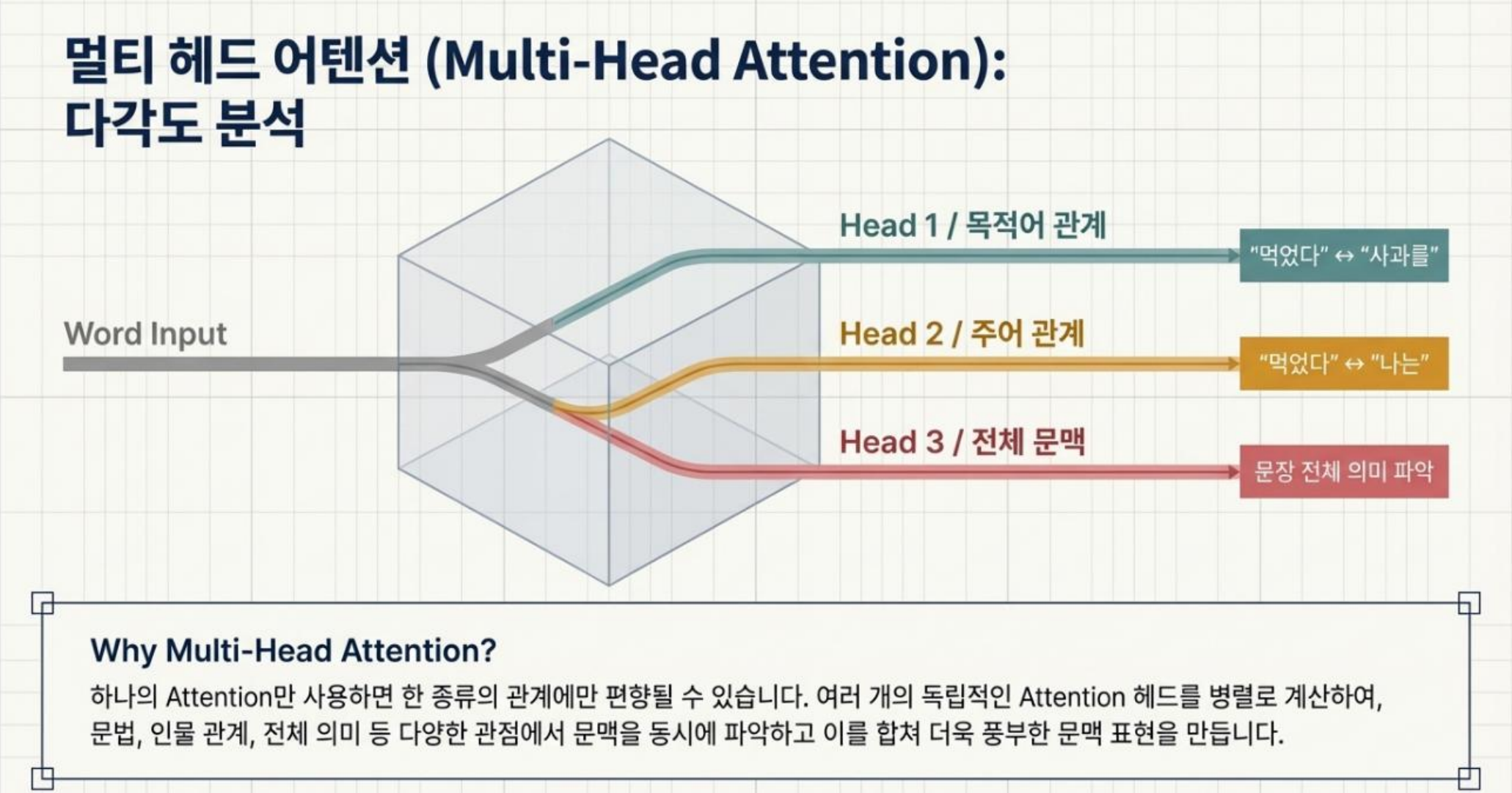
## 문장에서 중요한 Attention 관계

**예시 문장:** 철수는 사과를 먹었다. 그는 배가 고팠다.

| 토큰 | 주로 참고할 가능성이 큰 대상 | 의미 |
|---|---|---|
| 철수는 | 먹었다 | 철수가 행동의 주체임을 나타냅니다. |
| 사과를 | 먹었다 | 무엇을 먹었는지 연결합니다. |
| 먹었다 | 철수는, 사과를 | 누가 무엇을 먹었는지 문맥을 완성합니다. |
| 그는 | 철수는 | 대명사 그는이 철수를 가리킨다는 점을 반영합니다. |
| 배가 | 고팠다 | 상태 표현이 연결됩니다. |
| 고팠다 | 그는, 철수는 | 누가 배가 고픈 상태인지 연결합니다. |

**핵심 포인트**

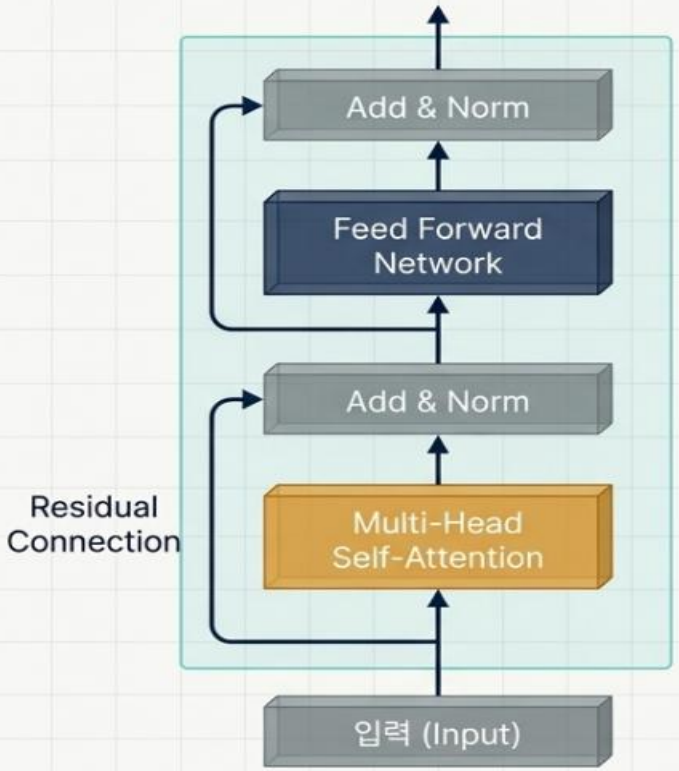
이 문장에서 트랜스포머의 핵심은 **그는 = 철수**라는 관계를 Self-Attention을 통해 반영하는 것입니다.




헤드(Head)란 정보를 바라보는 관점(독립된 처리 장치) 입니다.

**예시:** 영화 한 편을 볼 때,
**헤드 1:** "주인공의 감정선이 어떠한가?" (감정 담당)
**헤드 2:** "배경 음악과 영상미는 어떠한가?" (시각/청각 담당)
**헤드 3:** "스토리의 개연성은 맞는가?" (논리 담당)

# 인코더 (Encoder): 입력된 문장의 완벽한 이해




- **인코더의 역할:** 입력 문장 전체를 한 번에 읽고, 각 단어의 의미를 문맥(Context)이 완벽히 반영된 고밀도 벡터로 변환합니다.
- **Add (Residual Connection):** 원본 입력 정보를 그대로 더해주어 깊은 신경망에서도 정보 손실(기억 상실)을 방지합니다.
- **Norm (Layer Normalization):** 내부 값의 분포를 정규화하여 학습 과정을 빠르고 안정적으로 만듭니다.
- **Feed Forward:** 어텐션을 통해 파악된 정보를 확장하여 더 정교한 특징을 추출합니다.

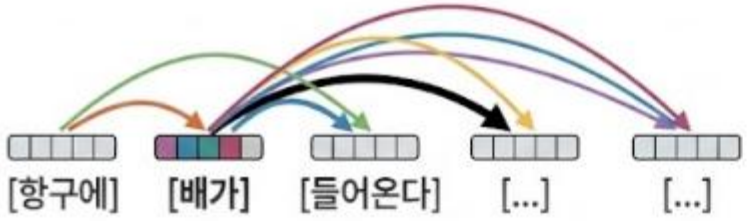
**Attention이 하는 일 (관계 파악)**
문장에서 "그(He)"라는 단어가 나왔을 때, 이 "그"가 앞에 나온 "철수"를 가리키는지 "영희"를 가리키는지 **주변 단어들과의 점수**를 매겨 연결합니다. 즉, 맥락(Context)을 만듭니다.

**Feed Forward가 하는 일 (특징 추출)**
어텐션이 "이 단어는 저 단어와 연결되어 있어!"라고 알려주면, **Feed Forward**는 그 정보를 바탕으로 **단어의 의미를 고차원적으로 변형**합니다.
활성화 함수를 사용하여 데이터를 복잡하게 비틀고 펼치면서, AI가 이해할 수 있는 더 정교한 특징(Feature)을 뽑아냅니다.
예) 항구에 배가 들어온다.
attention : "배"라는 단어 벡터에 "항구"와 관련된 정보가 조금 섞여 들어옵니다.
피드 포워드 : 어텐션이 가져온 '항구' 정보를 바탕으로 과일인 배가 아니라 운송 수단으로서의 배라고 확실한 특징(Feature)을 추출합니다.

## Attention: 관계의 파악 (수평적 정보 수집)

**"이 단어가 문장 속 다른 단어와 어떤 관계인가?"**

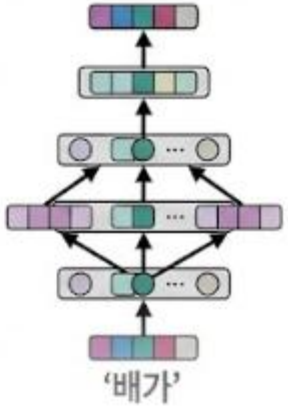
- **작동 원리:** 주변 단어들을 훑어 점수를 매기고, 맥락에 필요한 정보들을 긁어모음.
- **예시:** "항구에 배가 들어온다"에서 '배'가 '항구'와 관련 있다는 연결 고리를 발견.



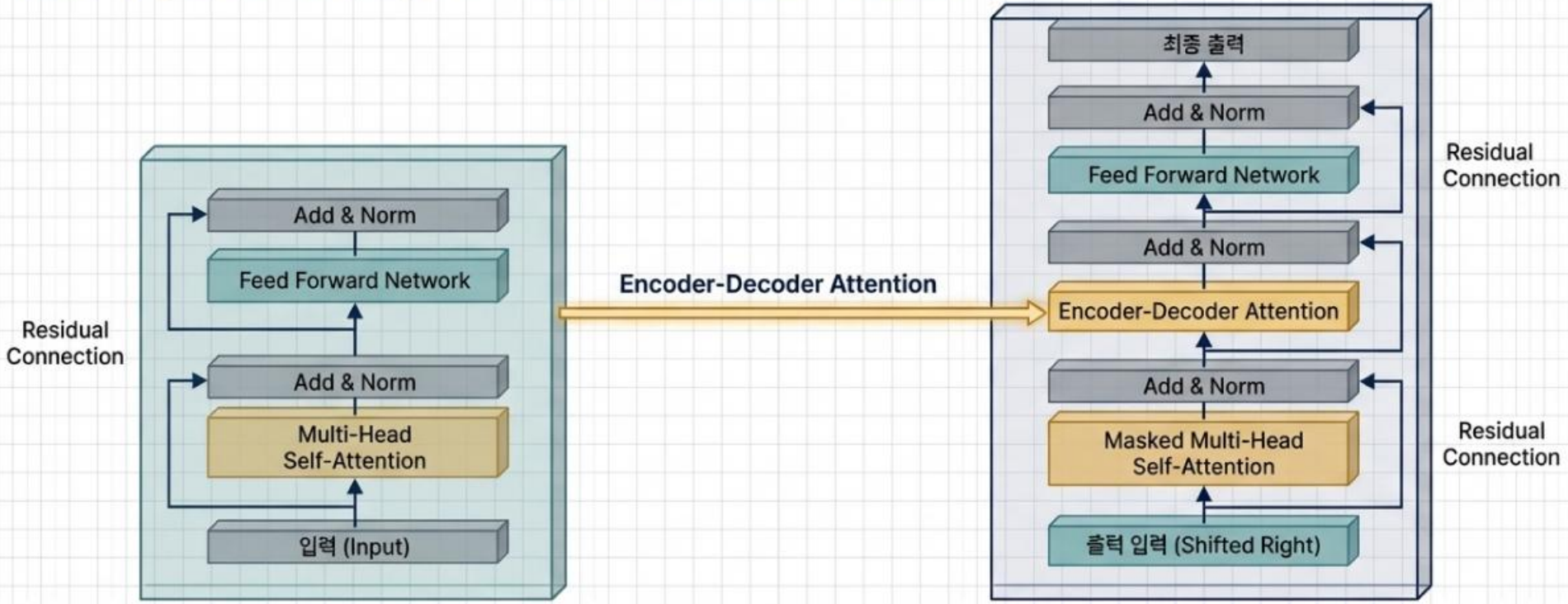

## Feed Forward: 의미의 심화 (수직적 특징 추출)

**"수집된 정보를 바탕으로 이 단어의 진짜 의미를 확정한다."**

- **작동 원리:** 어텐션이 가져온 맥락 데이터를 고차원 공간(Feature Space)으로 확장/변형 현하여 단어에 정체성을 부여.
- **예시:** '항구' 정보를 재료 삼아, '배'를 먹는 과일이 아닌 '운송 수단'이라는 구체적 특징으로 변환.



# 디코더 (Decoder): 문맥을 바탕으로 새로운 정보 생성



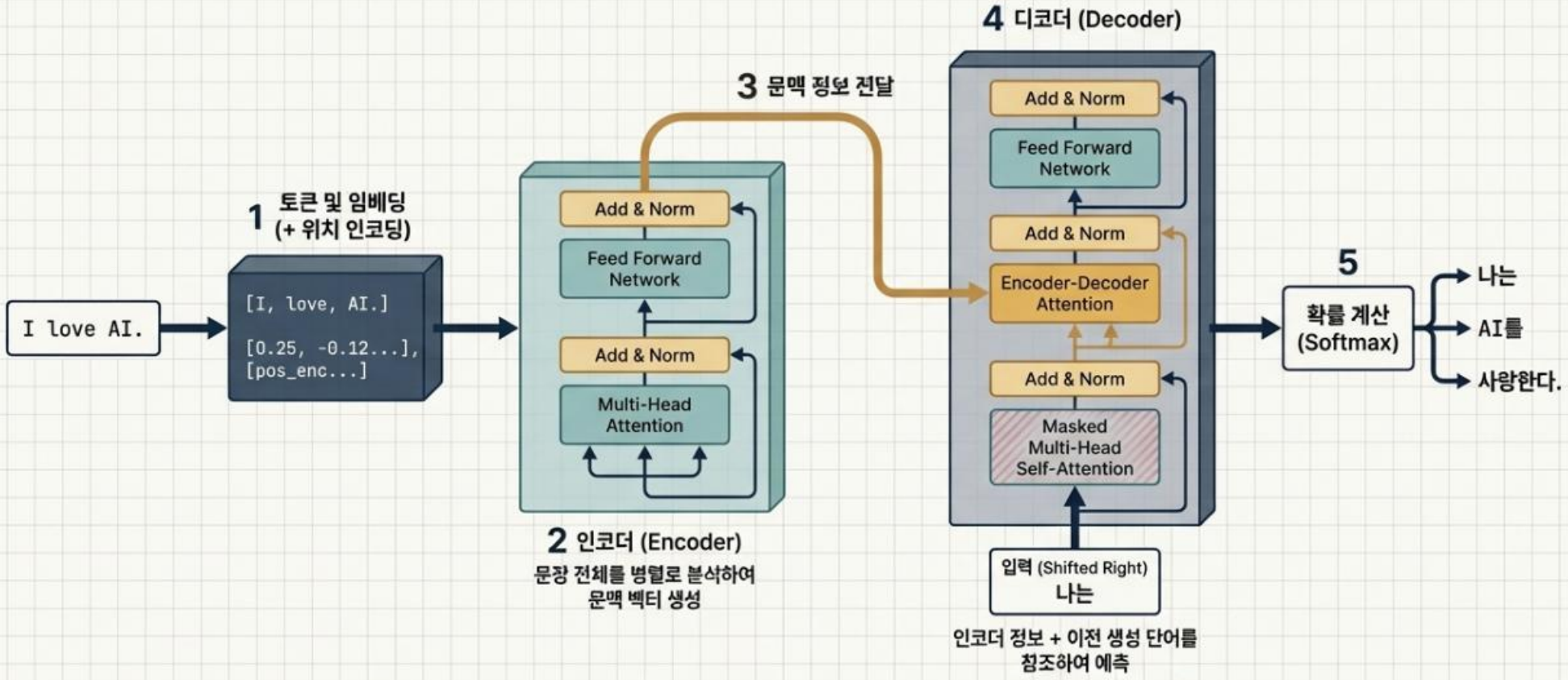

**디코더의 역할:** 인코더가 압축해준 '이해된 문맥 정보'와 '이전까지 자신이 생성한 단어들'을 바탕으로 다음 단어를 하나씩 순차적으로 예측하고 생성합니다.

**Encoder-Decoder Attention 메커니즘:**
- Query: 디코더의 현재 생성 중인 단어 상태.
- Key & Value: 인코더가 전달한 전체 문맥 정보.

# 디코더의 비밀: 마스크 (Mask) 장치

| | |
|---|---|
| **Look-Ahead Mask (미래 가리기)** | 디코더가 단어를 순차적으로 생성할 때, 정답을 미리 훔쳐보지 못하도록 미래의 단어를 Attention 연산에서 차단하여 모델 스스로 예측하게 만듭니다. |
| **Padding Mask (빈칸 가리기)** | 문장 길이를 맞추기 위해 채워넣은 의미 없는 빈칸(PAD) 토큰을 계산에서 제외시킵니다. |

# 종합: 트랜스포머의 전체 동작 흐름

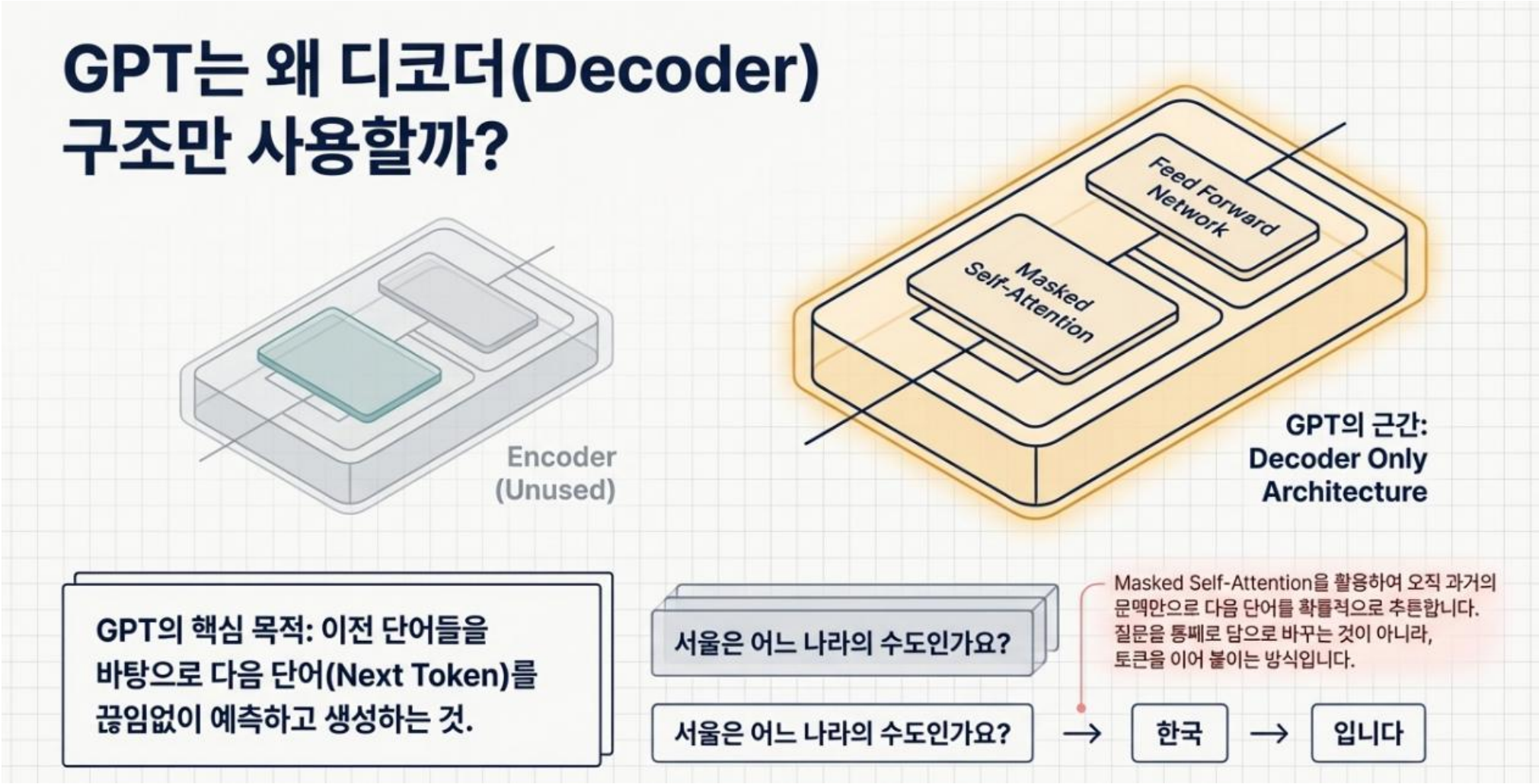



이 전체 과정을 문장이 끝날 때까지 반복 수행하여 최종 출력 문장을 완성합니다.

## 트랜스포머 단계별 처리과정

**예시 문장:** 철수는 사과를 먹었다. 그는 배가 고팠다.

| 단계 | 처리 내용 | 이 문장에서의 예시 | 결과 |
|---|---|---|---|
| 1 | 입력 문장 받기 | 철수는 사과를 먹었다. 그는 배가 고팠다. | 모델이 처리할 원문이 입력됩니다. |
| 2 | 토큰 분리 | [철수는] [사과를] [먹었다] [.] [그는] [배가] [고팠다] [.] | 문장이 토큰 단위로 나누어집니다. |
| 3 | 토큰 ID 변환 | [101, 205, 330, 18, 412, 509, 777, 18] | 각 토큰이 숫자 ID로 변환됩니다. |
| 4 | 단어 임베딩 | 철수는 → [0.12, -0.33, 0.81, ...] | 각 토큰이 의미 벡터로 바뀝니다. |
| 5 | 위치 벡터 생성 | position 1, position 2, position 3 ... | 순서 정보를 나타내는 벡터가 생성됩니다. |
| 6 | 단어 벡터 + 위치 벡터 | 철수는 임베딩 + 위치1 | 의미와 순서가 합쳐진 입력 벡터가 만들어집니다. |
| 7 | Q, K, V 생성 | 각 토큰마다 Query, Key, Value 생성 | Attention 계산을 위한 표현이 준비됩니다. |
| 8 | Self-Attention 계산 | 그는 → 철수는 를 강하게 참고 | 단어 간 관련도를 계산합니다. |
| 9 | 문맥 반영 | 그는 ≈ 철수를 가리키는 대명사 | 각 토큰이 문맥이 반영된 새 의미를 갖습니다. |
| 10 | Multi-Head Attention | 주어-동사, 대명사-대상 관계 등을 각각 파악 | 여러 관점에서 문장을 동시에 분석합니다. |
| 11 | Feed Forward Network | 각 토큰 벡터를 추가 변환 | 표현이 더 정교해집니다. |
| 12 | 층 반복 | 같은 블록을 여러 번 통과 | 문맥 이해가 점점 깊어집니다. |
| 13 | 최종 문맥 표현 생성 | 그는 = 철수 반영 | 문장 전체 의미를 반영한 최종 표현이 만들어집니다. |
| 14 | 작업 수행 | 질문 답변, 요약, 분류, 생성 | 예: 누가 배가 고팠나요? → 철수 |



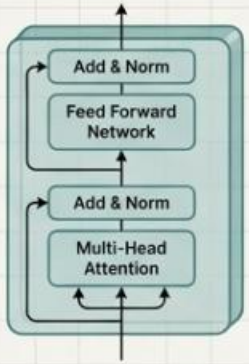

GPT(Generative Pre-trained Transformer)는 **'디코더 전용(Decoder-only)'** 모델입니다.
인코더가 아예 없기 때문에, 인코더로부터 데이터를 받던 **'Encoder-Decoder Attention' 층 자체가 통째로 사라집니다.**

**작동 방식:** 사용자가 입력한 질문("오늘 날씨 어때?")이 디코더의 첫 번째 입력으로 들어갑니다.

**Self-Attention의 역할:** 디코더 내부의 **Masked Self-Attention**이 입력된 질문의 단어들을 스스로 훑으면서 문맥을 파악합니다.

# 트랜스포머가 탄생시킨 AI 가계도

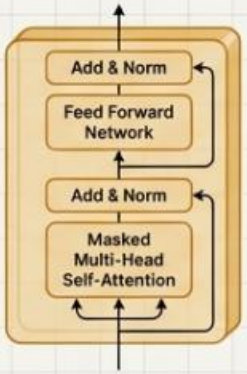
## BERT




### Encoder 기반 구조

- 문장을 양방향으로 완벽히 이해하는 데 특화.
- 주요 용도: 문장 분류, 감성 분석, 검색 엔진.

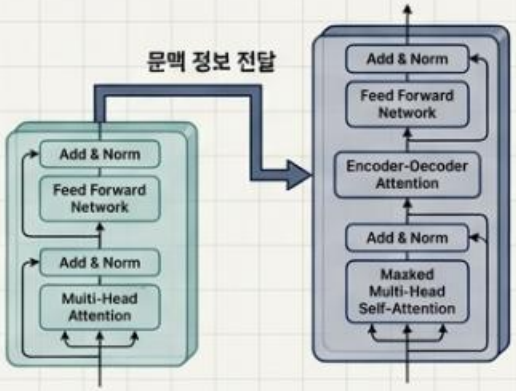
## GPT




### Decoder 기반 구조

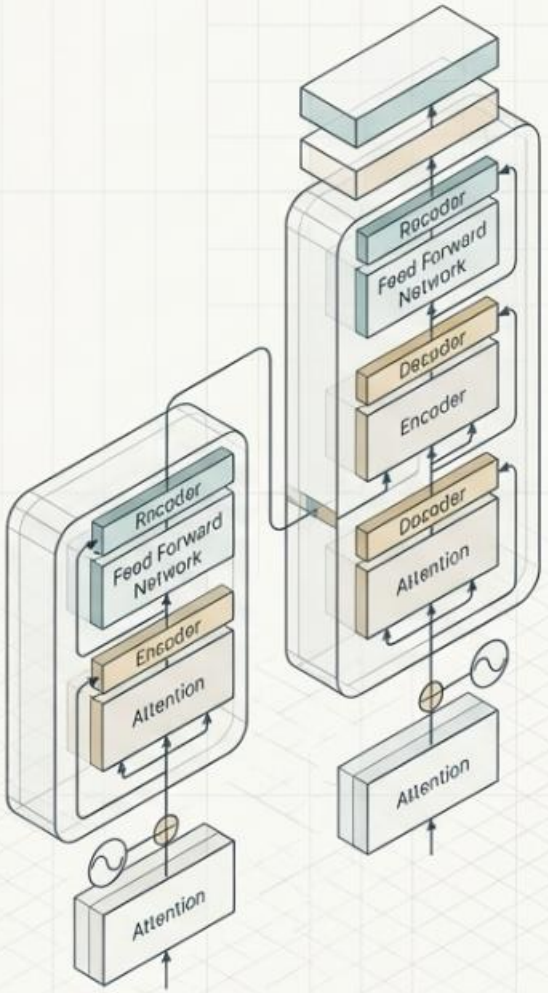
- 단어를 순차적으로 매끄럽게 생성하는 데 특화.
- 주요 용도: 챗봇, 텍스트 생성, 코드 작성.

## T5




### Encoder-Decoder 통합 구조

- 입력과 출력이 모두 텍스트인 변환 작업에 특화.
- 주요 용도: 기계 번역, 문서 요약.

# 현대 딥러닝의 핵심, 5가지 요약

1. **병렬 처리의 혁명:** RNN의 순차 처리 한계를 극복하고, 문장 전체를 한 번에 연산하여 압도적 성능 달성.
2. **Self-Attention 엔진:** 단어 간의 모든 관계를 직접 계산하여 거리와 무관하게 정확한 문맥을 파악.
3. **데이터 처리:** 모든 텍스트는 [토큰화] → [임베딩] → [위치 인코딩] 과정을 거쳐 컴퓨터용 데이터로 변환.
4. **이해와 생성의 분리:** 문맥을 파악하는 '인코더' (Encoder)와 미래를 마스킹하여 순차 예측하는 '디코더' (Decoder)의 결합.
5. **LLM의 절대적 기반:** 현재 우리가 사용하는 BERT, GPT, T5 등 모든 대형 언어 모델의 근간.